

1장 머신러닝과 딥러닝 , 2장 실습 환경 설정과 파이토치 기초

1장 머신러닝과 딥러닝

1.1 인공지능, 머신러닝과 딥러닝

인공지능 : 인간의 지능을 모방하여 사람이 하는 일을 컴퓨터가 할 수 있도록 하는 기술 (머신러닝과 딥러닝으로 구현 가능)

머신러닝과 딥러닝 모두 학습 모델을 제공하여 데이터를 분류할 수 있는 기술이지만 접근 방식에 차이가 있음

머신러닝 : 주어진 데이터를 인간이 먼저 전처리해야함, 데이터의 특징을 스스로 추출하지 못함

딥러닝 : 대량의 데이터를 신경망에 적용하면 컴퓨터가 스스로 분석한 후 답을 찾음

	머신러닝	딥러닝
원리	입력 데이터에 알고리즘을 적용하여 예측을 수행	정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석
재사용	입력 데이터를 분석하기 위해 다양한 알고리즘 사용 동일한 유형의 데이터 분석을 위한 재사용은 불가능	구현된 알고리즘은 동일한 유형의 데이터를 분석하는데 재사용 가능
데이터	수천 개의 데이터 필요	수백만 개 이상의 데이터 필요
훈련 시간	단시간	장시간
결과	점수 또는 분류 등 숫자 값	점수, 텍스트, 소리 등 모두 가능

1.2 머신러닝이란

컴퓨터가 학습할 수 있게 알고리즘과 기술을 개발하는 분야

머신러닝의 학습 단계에서는 훈련 데이터를 머신러닝 알고리즘에 적용하여 학습시키고, 학습 결과로 모형이 생성됨

예측 단계에서는 학습 단계에서 생성된 모형에 새로운 데이터를 적용하여 결과를 예측함

머신러닝의 주요 구성 요소는 데이터와 모델(모형)

- 데이터 : 머신러닝이 학습 모델을 만드는 데 사용하는 것, 실제 데이터의 특징이 잘 반영되고 편향되지 않는 훈련 데이터를 확보하는 것이 중요함
- 모델 : 머신러닝의 학습 단계에서 얻은 최종 결과물(가설), 모델선택 → 모델 학습 및 평가 → 평가를 바탕으로 모델 업데이트 단계를 반복하며 주어진 문제를 가장 잘 풀 수 있는 모델을 찾음

머신러닝의 학습 알고리즘으로는 지도, 비지도, 강화 학습이 있음

- 지도 학습 : 정답이 무엇인지 컴퓨터에 알려주고 학습시키는 방법
- 비지도 학습 : 정답을 알려주지 않고 특징이 비슷한 데이터를 클러스터링하여 예측하는 학습 방법
- 강화 학습 : 자신의 행동에 대한 보상을 받으며 학습을 진행 (보상이 커지는 행동은 자주 하도록 하고, 줄어드는 행동은 덜 하도록 하여 학습을 진행시킴)

1.3 딥러닝이란

인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 머신러닝 방법의 일종

인간의 뇌를 기초로 하여 설계해서 컴퓨터에 뉴런과 시냅스 개념을 적용함

각각의 뉴런은 복잡하게 연결된 수많은 뉴런을 병렬 연산하여 기존에 컴퓨터가 수행하지 못했던 음성, 영상 인식 등 처리를 가능하게 함

딥러닝의 학습 과정

데이터 준비 → 모델 정의 → 모델 컴파일 (옵티마이저, 손실함수 선택) → 모델 훈련 → 모델 예측

- 데이터 준비 : 파이토치나 케라스에서 제공하는 데이터셋 사용 (이미 전처리 됨) 아니면 캐글에서 공개된 데이터 사용
- 모델 정의 : 신경망을 생성 (은닉층 개수 등 결정)
- 모델 컴파일 : 활성화 함수, 손실 함수, 옵티마이저 선택 (과적합을 피할 수 있는 활성화 함수 및 옵티마이저 선택이 중요)
- 모델 훈련 : 한 번에 처리할 데이터 양을 지정, 전체 훈련 데이터셋에서 일정한 묶음으로 나누어 처리할 수 있는 배치와 훈련의 횟수인 에포크 선택이 중요, 훈련 과정에서 값의 변화를 시각적으로 표현하여 눈으로 확인하면서 파라미터와 하이퍼파라미터에 대한 최적의 값을 찾아야 함
- 모델 예측 : 검증 데이터셋을 생성한 모델에 적용하여 실제로 예측을 진행

*훈련 데이터셋 1000개에 대한 배치 크기가 20 : 데이터 20개마다 모델 가중치를 한번씩 업데이트 (총 50번의 가중치가 업데이트 됨)

→ 에포크가 10, 배치 크기가 20 : 가중치를 50번 업데이트하는 것을 총 열번 반복 (가중치가 총 500번 업데이트 됨)

가중치 값을 업데이트하기 위한 역전파가 중요함

역전파 계산 과정에서 사용되는 미분이 성능에 영향을 미치는 주요한 요소

딥러닝은 심층 신경망을 사용한다는 점에서 머신러닝과 차이가 있음

심층 신경망은 은닉층이 두 개 이상인 신경망으로 데이터셋의 어떤 특성들이 중요한지 스스로에게 가르쳐 줄 수 있는 기능이 있음

딥러닝은 활용 분야에 따라 지도, 비지도, 전이 학습으로 분류됨

	유형	알고리즘
지도 학습	이미지 분류	CNN AlexNet ResNet
	시계열 데이터 분석	RNN LSTM
비지도 학습	군집	가우시나 혼합 모델 자기 조직화 지도
	차원 축소	오토인코더 주성분 분석
전이 학습	전이 학습	버트 MobileNetV2
강화 학습		마르코프 결정 과정

지도 학습

• 이미지 분류 (합성곱 신경망)

이미지 또는 비디오 상의 객체를 식별하는 컴퓨터 비전 기술

컴퓨터 비전에서 가장 많이 사용되는 것은 '합성곱 신경망'

합성곱 신경망은 목적에 따라 이미지 분류, 이미지 인식, 이미지 분할로 분류 가능

- 이미지 분류 : 이미지를 알고리즘에 입력하면 그 이미지가 어떤 클래스에 속하는지 알려줌 (이미지 데이터를 유사한 것 끼리 분류할 때 사용)
- 이미지 인식 : 사진을 분석하여 그 안에 있는 사물의 종류를 인식하는 것
- 이미지 분할 : 사물이나 배경 등 객체 간 영역을 픽셀 단위로 구분하는 기술

• 시계열 데이터 분류 (순환 신경망)

시간에 따른 데이터가 있을 때 '순환 신경망' 사용

하지만 역전파 과정에서 기울기 소멸 문제가 발생하는 단점이 존재

→ 게이트를 세 개 추가한 LSTM으로 문제점을 개선

망각 게이트 (과거 정보 잊기), 입력 게이트 (현재 정보 기억), 출력 게이트 (최종 결과)를 도입하여 기울기 소멸 문제 해결하여 시계열 문제에서 활발히 사용됨

비지도 학습

- 워드 임베딩

사람의 언어를 컴퓨터가 이해하고 처리하게 하려면 변환이 필요

워드 임베딩을 통해 단어를 벡터로 표현함

단어의 의미를 벡터화하는 워드투벡터와 글로브를 가장 많이 사용

- 군집

아무런 정보가 없는 상태에서 데이터를 분류하는 방법

한 클러스터 안의 데이터는 매우 비슷하게 구성하고 다른 클러스터의 데이터와 구분되도록 나눔

전이 학습

사전에 학습이 완료된 모델을 가지고 우리가 원하는 학습에 미세 조정 기법을 이용하여 학습 시키는 방법

사전에 학습이 완료된 모델이 필요하며, 그 모델을 어떻게 활용하는지에 대한 접근 방법이 필요

사전 학습 모델 : 풀고자 하는 문제와 비슷하면서 많은 데이터로 이미 학습이 되어 있는 모델
VGG, 인셉션, MobileNet과 같은 사전 학습 모델을 사용하면 효율적인 학습이 가능함

2장 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

딥러닝 프레임워크로 루아 언어로 개발되었던 토치를 파이썬 버전으로 내놓은 것

토치는 파이썬의 넘파이 라이브러리처럼 과학 연산을 위한 라이브러리로 공개되었지만 이후 딥러닝 프레임워크로 발전함

간결하고 빠른 구현성을 가짐

GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크

- GPU : 연산 속도를 빠르게 함

- 텐서 : 파이토치의 데이터 형태, 단일 데이터 형식으로 된 자료들의 다차원 행렬

- 동적 신경망 : 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망, 모델의 네트워크 조 작이 가능함

*텐서는 행렬의 다차원 표현, 같은 크기의 행렬이 여러 개 묶여 있는 것으로 표현할 수 있음

파이토치의 장점에는 효율적인 계산, 낮은 CPU 활용, 직관적인 인터페이스 등이 있음

- 효율적인 계산 (단순함) : 파이썬 환경과 쉽게 통합 가능, 디버깅이 직관적이고 간결함
- 낮은 CPU 활용 (성능) : 모델 훈련을 위한 CPU 사용률이 텐서플로에 비해 낮음, 학습 및 추론 속도가 빠르고 다루기 쉬움
- 직관적인 인터페이스 : 텐서플로처럼 잦은 API 변경이 없어 배우기 쉬움

파이토치의 아키텍처는 크게 세 개의 계층으로 나누어짐

파이토치 API - 파이토치 엔진 - 연산 처리

- 파이토치 API

가장 상위 계층으로 사용자가 이해하기 쉬운 API를 제공하여 텐서에 대한 처리와 신경망을 구축하고 훈련할 수 있도록 도움

사용자 인터페이스를 제공하지만 실제 계산은 수행하지 않음

C++로 작성된 파이토치 엔진으로 그 작업을 전달하는 역할만 함

사용자의 편의성을 위해 패키지들을 제공함

- torch : GPU를 지원하는 텐서 패키지
- torch.autograd : 자동 미분 패키지
- torch.nn : 신경망 구축 및 훈련 패키지
- torch.multiprocessing : 파이썬 멀티프로세싱 패키지
- torch.utils : DataLoader 및 기타 유틸리티를 제공하는 패키지

- 파이토치 엔진

파이토치 API 아래에 위치한 계층

Autograd C++, Aten C++, JIT C++, Python API로 구성됨

- Autograd C++ : 가중치, 바이어스를 업데이트하는 과정에서 필요한 미분을 자동으로 계산
- Aten C++ : C++ 텐서 라이브러리 제공
- JIT C++ : 계산을 최적화하기 위한 JIT 컴파일러

- 연산 처리

텐서에 대한 연산을 처리하는 계층

가장 아래 계층에 속하는 C 또는 CUDA 패키지는 상위의 API에 할당된 거의 모든 계산을 수행

텐서를 메모리에 저장할 때는 1차원 배열 형태가 됨 (1차원 배열 형태여야만 메모리에 저장 가능)

변환된 1차원 배열을 storage (스토리지)라고 함

offset(오프셋)은 텐서에서 첫 번째 요소가 스토리지에 저장된 인덱스

stride(스트라이드)는 각 차원에 따라 다음 요소를 얻기 위해 건너뛰기가 필요한 스토리지의 요소 개수

A, A^T 는 전치 행렬로 1차원 배열 형태로 변환하게 되면 다른 형태를 갖지만 스토리지의 값들은 서로 같음

→ 이 둘을 구분하기 위해 오프셋과 스트라이드를 사용

스트라이드 = (다음 행까지 건너뛸 칸 수, 다음 열까지 건너뛸 칸 수) (첫번째 요소 기준)

2.2 파이토치 기초 문법

파이토치를 사용하기 위해서는 텐서의 사용법을 잘 숙지해야함

텐서를 잘 다룰 수 있어야 문제 해결이 가능하고 신경망에서 데이터 입력과 출력도 제어할 수 있음

데이터 준비

데이터를 호출할 때는 파이썬 라이브러리나 파이토치에서 제공하는 데이터를 이용하는 방법이 있음

데이터가 이미지일 경우 분산된 파일에서 데이터를 읽은 후 전처리를 하고 배치 단위로 분할하여 처리

텍스트일 경우 임베딩 과정을 거쳐 서로 다른 길이의 시퀀스를 배치 단위로 분할하여 처리

- 단순하게 파일을 불러와서 사용

- 커스텀 데이터셋을 만들어서 사용 : 데이터를 한번에 다 부르지 않고 조금씩 나누어 불러서 사용

- 파이토치에서 제공하는 데이터셋 사용

모델 정의

파이토치에서 모델을 정의하기 위해서는 모듈을 상속한 클래스를 사용

- 계층 : 모듈 또는 모듈을 구성하는 한 개의 계층 (합성곱층, 선형계층 등)

- 모듈 : 한 개 이상의 계층이 모여서 구성된 것으로 모듈이 모여 새로운 모듈을 만들 수 있음

- 모델 : 최종적으로 원하는 네트워크로 한 개의 모듈이 모델이 될 수 있음

- 단순 신경망 정의 : nn.Module을 상속받지 않는 매우 단순한 모델, 구현이 쉬움
- nn.Module()을 상속하여 정의
- Sequential 신경망을 정의
- 함수로 신경망 정의

모델의 파라미터 정의

모델을 학습 시키기 전에 필요한 파라미터들을 정의함

- 손실함수 : 학습하는 동안 출력과 실제 값 사이의 오차를 측정, 오차를 구해서 모델의 정확성을 측정
- 옵티마이저 : 데이터와 손실 함수를 바탕으로 모델의 업데이트 방향 결정
- 학습률 스케줄러 : 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소시킴, 학습 초기에는 빠른 학습을 진행하다가 전역 최소점 근처에 다다르면 학습률을 줄여서 최적점을 찾아갈 수 있도록 해줌
- 지표 : 훈련과 테스트 단계를 모니터링함

모델 훈련

앞에서 준비한 데이터로 모델을 학습시킴

파라미터 w 와 b 에 임의의 값을 적용하여 시작하며 오차가 줄어들어 전역 최소점에 이를 때까지 파라미터(w, b)를 계속 수정함

먼저 메서드를 이용하여 기울기를 초기화함

누적 계산이 필요하지 않을 때에는 입력 값을 모델에 적용하기 전에 미분 값이 누적되지 않게 초기화해야함

다음은 메서드를 이용하여 기울기를 자동 계산함

모델 평가

주어진 테스트 데이터셋을 사용하여 모델을 평가함

함수와 모듈을 이용하는 두가지 방법이 존재

훈련 과정 모니터링

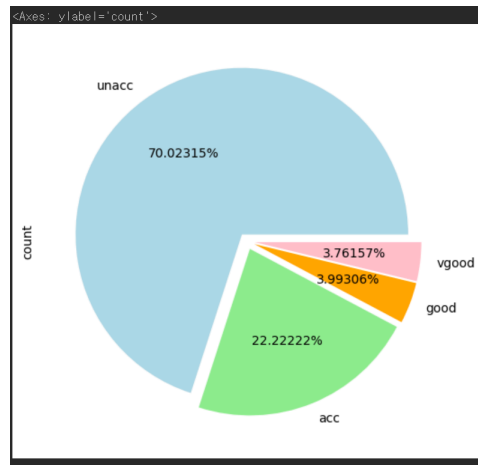
텐서보드를 이용하면 학습에 사용되는 각종 파라미터 값이 어떻게 변화하는지 손쉽게 시각화하여 살펴볼 수 있음

텐서보드 설정 → 텐서보드에 기록 → 텐서보드를 사용하여 모델 구조를 살펴봄

2.4 파이토치 코드 맛보기

	price	maint	doors	persons	lug_capacity	safety	output
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

pd.read_csv()를 통해 다운로드 받은 데이터 파일을 불러옴
 데이터 프레임 내의 처음 n줄을 출력해서 데이터의 내용을 확인할 수 있음



주어진 데이터셋을 이해하기 쉽도록 분포 형태로 시각화하여 표현
 대부분의 자동차는 허용 불가능한 상태에 있고 20%만 허용이 가능한 수준

```
tensor([[3, 3, 0, 0, 2, 1],
        [3, 3, 0, 0, 2, 2],
        [3, 3, 0, 0, 2, 0],
        [3, 3, 0, 0, 1, 1],
        [3, 3, 0, 0, 1, 2],
        [3, 3, 0, 0, 1, 0],
        [3, 3, 0, 0, 0, 1],
        [3, 3, 0, 0, 0, 2],
        [3, 3, 0, 0, 0, 0],
        [3, 3, 0, 1, 2, 1]])
```

분석하기 좋게 데이터를 전처리해야함
 범주 특성을 갖는 데이터를 범주형 타입으로 변환
 파이토치를 이용한 모델 학습을 해야하므로 범주형 타입을 모두 텐서로 변환해야함
 범주형 데이터를 텐서로 변환하기 위한 절차
 범주형 데이터 → dataset[category] → 넘파이 배열 → 텐서
 워드 임베딩은 유사한 단어끼리 유사하게 인코딩되도록 표현하는 방법
 높은 차원의 임베딩일수록 단어 간의 세부적인 관계를 잘 파악할 수 있음
 따라서 단일 숫자로 변환된 넘파이 배열을 N차원으로 변경하여 사용

```
1383
1383
345
345
```

데이터 셋을 훈련과 테스트 용도로 분리

데이터 준비가 끝나면 모델의 네트워크를 생성

for문을 이용하여 각 계층을 all_layers 목록에 추가

- linear : 선형 계층 (입력 데이터에 선형 변환을 진행한 결과)
- ReLU : 활성화 함수로 사용
- BatchNorm1d : 배치 정규화 용도로 사용
- Dropout : 과적합 방지에 사용

모델을 훈련시키기 전에 손실함수와 옵티마이저 정의

데이터를 분류해야하므로 크로스 엔트로피 손실함수와 아담 옵티마이저 사용

```
epoch: 1 loss: 1.54789495
epoch: 26 loss: 1.37239766
epoch: 51 loss: 1.30173457
epoch: 76 loss: 1.18175280
epoch: 101 loss: 1.07668245
epoch: 126 loss: 0.93468064
epoch: 151 loss: 0.81955642
epoch: 176 loss: 0.76836622
epoch: 201 loss: 0.69586188
epoch: 226 loss: 0.66175497
epoch: 251 loss: 0.63822299
epoch: 276 loss: 0.61097032
epoch: 301 loss: 0.61597914
epoch: 326 loss: 0.60171980
epoch: 351 loss: 0.59838820
epoch: 376 loss: 0.58237243
epoch: 401 loss: 0.58972049
epoch: 426 loss: 0.58324713
epoch: 451 loss: 0.57105809
epoch: 476 loss: 0.57326287
epoch: 500 loss: 0.5703880787
```

모델을 훈련시키면 25 에포크마다 오차 정보를 출력함

```
tensor([[ 0.9937,  0.7440, -1.1276, -0.8158],
        [ 2.4725,  1.4238, -3.4774, -3.7849],
        [ 1.2973,  0.6527, -2.4388, -2.3937],
        [ 3.1217,  1.5705, -4.2946, -4.2357],
        [ 1.3613,  0.8988, -1.9114, -1.8422]])
```

[[258 1]					
[86 0]]					
		precision	recall	f1-score	support
	0	0.75	1.00	0.86	259
	1	0.00	0.00	0.00	86
	accuracy			0.75	345
	macro avg	0.38	0.50	0.43	345
	weighted avg	0.56	0.75	0.64	345
0.7478260869565218					

훈련시킨 모델로 예측 수행

테스트 용도의 데이터셋에 대한 손실 값을 보여줌

loss값이 훈련 데이터셋에서 도출된 loss값과 비슷하므로 과적합이 발생하지 않았다고 판단 가능

모델 평가를 실행한 결과가 출력됨

accuracy, precision, recall, f1-score 값을 보여줌

TP (True Positive) : 모델이 1로 예측했는데 실제 값도 1인 경우

TN (True Negative) : 모델이 0으로 예측했는데 실제 값도 0인 경우

FP (False Positive) : 모델이 1로 예측했는데 실제 값이 0인 경우 (Type 1 오류)

FN (False Negative) : 모델이 0으로 예측했는데 실제 값이 1인 경우 (Type 2 오류)

정확도 (accuracy) : 전체 예측 건수에서 정답을 맞힌 건수의 비율

$$\frac{TP+TN}{TP+TN+FP+FN}$$

재현율 (recall) : 실제로 정답이 1일 때 모델도 1로 예측한 비율

$$\frac{TP}{TP+FN}$$

정밀도 (precision) : 모델이 1이라고 예측한 것 중에서 실제로 정답이 1인 비율

$$\frac{TP}{TP+FP}$$

F1-스코어 : 정밀도와 재현율의 조화 평균 (정밀도와 재현율은 trade-off 관계)

$$2 * \frac{precision*recall}{precision+recall}$$